UNIVERSIDAD TECNOLÓGICA NACIONAL · FACULTAD REGIONAL CÓRDOBA

INGENIERÍA Y CALIDAD DE SOFTWARE (ISW)
Resumen Institucional Consolidado — Primer Parcial

Departamento de Ingeniería en Sistemas de Información · Cátedra de Ingeniería y Calidad de Software · Ciclo Lectivo 2026

Propósito del Documento: Compendio académico y técnico completo de la totalidad de los contenidos teóricos y prácticos evaluados en la primera instancia parcial de la asignatura (Notebook Oficial ISW — UTN FRC), estructurado y nominado bajo las normas de Gestión de Configuración de Software (SCM) del repositorio oficial de cátedra **ISW_4K1_G1_2026**.

Ficha de Identificación del Ítem de Configuración (Matriz de ICs)

Atributo de Configuración	Especificación Institucional
Repositorio SCM	`ISW_4K1_G1_2026` (Curso: 4K1 · Comisión/Grupo: Grupo 1 · Año: 2026)
Tipo de Ítem / Clasificación	`produccion_propia` — Ítem de Configuración: **Resumen Consolidado de Cátedra**
Regla de Nombrado Aplicada	`resumen-u<numero_unidad>-<tema>-<ApellidoAutor_NombreAutor>.<ext>`
Nombre Físico del Archivo (Grupal / Individual)	`resumen-u01-primerparcial-grupo1.pdf` (Equivalente individual: `resumen-u01-primerparcial-Duarte_Cayetano.pdf`)
Ubicación Física en el Repositorio	`ISW_4K1_G1_2026/produccion_propia/resumenes/`
Línea Base (Git Tag Anotado)	`lb-parcial-01` (Hito: Evaluación Formal Primer Parcial — Unidades 01, 02 y 03)
Autor / Responsable de Elaboración	Cayetano Amaro Duarte (`Duarte_Cayetano` / `grupo1`)
Estado de Configuración	Versión Definitiva Consolidada (Verificada por Peer Review, sin marcas de borrador)

Alcance Temático Evaluado en el Primer Parcial

Unidad	Título Oficial según Programa 2026	Ejes Conceptuales y Prácticos Incluidos
Unidad 01	Ingeniería de Software en Contexto (Completa)	Definición y capas de la ISW, Crisis del Software, Paper *No Silver Bullet* (Fred Brooks: Esencia vs. Accidente), Disciplinas, Proyectos fallidos/exitosos, Marco Cynefin, Procesos Definidos vs. Empíricos, Modelos de Ciclo de Vida, Componentes P4 y Triple Restricción.
Unidad 02	Gestión Lean-Ágil de Productos de Software (Parte I)	Gestión de Producto (Visión, Discovery vs. Delivery, UVP, MVP, MMP, MMR, MMF), Requerimientos Ágiles y User Stories (3C, INVEST, Slicing, Criterios de Aceptación, Spikes), Estimaciones Tradicionales y Ágiles (Story Points, Fibonacci, Planning Poker, Velocidad) y Framework Scrum (Roles, Eventos, Artefactos, DoR vs. DoD, Métricas).
Unidad 03	Gestión del Software como Producto (Parte I: SCM)	Administración de Configuración de Software (SCM), Ítems de Configuración (IC), Versión, Variante, Configuración, Líneas Base (Baselines), Actividades (Identificación, Control de Versiones/Cambios y CCB, Auditorías FCA/PCA, Informes de Estado) y SCM en entornos Ágiles (Git).
Práctico 1P	Resolución de Caso de Estudio (Product Owner & SCM)	Algoritmo de definición y justificación del MVP, redacción de tarjeta completa de User Story con Criterios de Aceptación y Pruebas de Usuario, estimación con US Canónica y diseño de matriz de ICs.

Fuentes Oficiales Trazables (Notebook ISW): `2026 - ICS - Planificación Ingeniería y Calidad de Software.pdf`, `Temas que se evalúan en el primer parcial`, `Guia_ISW_1er_Parcial.docx`, `01 Introducción a la Ingenieria de Software.pdf`, `02 SCM.pdf`, `03 Rqs Agiles USER STORIES.pdf`, `04 Estimaciones Ágiles.pdf`, `05 Gestión de Productos.pdf`, `06 Componentes de Proyecto SW.pdf`, `07 Estimaciones de Sw.pdf`, `08 SCRUM 2026.pdf`, `Cynefin Mini book.pdf` y `RESUMEN FINAL -ISW- ALEX.pdf`.

1. Unidad 01: La Ingeniería de Software en Contexto y Naturaleza del Software

Fuentes de cátedra (Notebook ISW): `01 Introducción a la Ingenieria de Software.pdf`, `Ingenieria de Software - Ian Somerville - 9 edicion.pdf`, `ingenieria del software - roger pressman 7ma edición.pdf`, `Guia_ISW_1er_Parcial.docx`.

1.1 Definiciones Fundamentales, Naturaleza del Producto y Capas de la ISW

El **software** comprende: (1) las **instrucciones** (programas de computadora) que al ejecutarse proporcionan las características, funciones y el grado de desempeño deseados; (2) las **estructuras de datos** que permiten a los programas manipular información adecuadamente; y (3) la **información**

descriptiva (documentación) tanto en papel como en formas virtuales que describe la operación y el uso de los programas.

- **Diferencias entre el Software y los Productos Industriales o de Hardware:**
  - **El software es intangible y lógico, no físico:** El costo más significativo del software reside en la ingeniería y el esfuerzo intelectual humano de desarrollo, no en la copia o fabricación física.
  - **El software se desarrolla o modifica mediante ingeniería, no se fabrica en el sentido clásico:** No existe producción en masa porque cada producto de software resuelve un problema distinto; una vez escrito el código, producir copias adicionales tiene costo marginal casi nulo.
  - **El software no se "desgasta" con el uso, pero sí se deteriora con los cambios:** Mientras que el hardware exhibe la clásica "curva de la bañera" (alta tasa de fallas iniciales, meseta de vida útil y desgaste físico final por fatiga de materiales), el software no sufre desgaste ambiental. Sin embargo, cada cambio o mantenimiento introduce nuevos defectos que elevan la tasa de fallas ("curva real deteriorada") hasta volverlo inmantenible si no se gestiona su calidad y configuración.
- **Definiciones Clásicas de Ingeniería de Software (ISW):**
  - **Fritz Bauer (Conferencia OTAN, 1969):** *"El establecimiento y uso de principios sólidos de la ingeniería con el fin de obtener económicamente software que sea confiable y que funcione eficientemente sobre máquinas reales"*.
  - **Estándar IEEE 610.12:** *"(1) La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software. (2) El estudio de enfoques como en (1)"*.
  - **Ian Sommerville:** *"Disciplina de la ingeniería que comprende todos los aspectos de la producción de software desde las etapas iniciales de la especificación del sistema hasta el mantenimiento de éste después de que se utiliza"* (abarca procesos técnicos, administración de proyectos y desarrollo de herramientas/métodos).
- **Las 4 Capas de la Ingeniería de Software (Pressman):** La ISW es una tecnología estratificada donde: (1) el **Compromiso con la Calidad** constituye el cimiento organizacional; (2) la capa de **Proceso** es el adhesivo que mantiene unidas las capas tecnológicas y permite el desarrollo racional y oportuno; (3) los **Métodos** indican "cómo" construir técnicamente el software (análisis, diseño, codificación, pruebas); y (4) las **Herramientas** brindan soporte automatizado o semiautomatizado al proceso y a los métodos (CASE, IDEs, SCM).

1.2 La Crisis del Software, Proyectos Fallidos/Exitosos y Disciplinas

El término **Crisis del Software** fue acuñado en la Conferencia de la OTAN en Garmisch (Alemania, 1968) para describir la incapacidad de la industria de producir software a medida que la potencia del hardware crecía exponencialmente. Sus síntomas históricos y actuales (reflejados en el *Standish Group Chaos Report*) incluyen: estimaciones de plazo y presupuesto sistemáticamente excedidas (proyectos *challenged*), baja calidad y confiabilidad en producción, software que no cumple los requerimientos reales del usuario, código inmantenible y proyectos cancelados antes de su entrega (proyectos *impaired/failed*).

- **Causas de Falla vs. Factores de Éxito en Grandes Proyectos:** Los grandes fracasos históricos (como el sistema de despacho de ambulancias de Londres LAS, el sistema de equipajes del Aeropuerto de Denver o el FBI Virtual Case File) se explican por requerimientos incompletos o cambiantes sin gestión, falta de involucramiento del usuario, estimaciones irreales impuestas por presión comercial y

ausencia de control de calidad. Por el contrario, los factores de éxito exigen involucrar al usuario desde el inicio, apoyo ejecutivo, requerimientos priorizados por valor, entregas incrementales tempranas, manejo realista de expectativas y monitoreo continuo.

- **Disciplinas que conforman la Ingeniería de Software:** Se dividen en **Disciplinas Técnicas o de Ingeniería** (Requerimientos, Análisis y Diseño Arquitectónico, Construcción/Implementación, Pruebas/Testing y Despliegue) y **Disciplinas de Gestión y Soporte** (Gestión de Proyectos, Gestión de Configuración de Software - SCM, Aseguramiento de la Calidad - SQA/PPQA y Gestión de Procesos).

1.3 El Artículo *"No Silver Bullet: Essence and Accidents of Software Engineering"* (Fred Brooks, 1986)

Fuentes de cátedra (Notebook ISW): `Explicación de No Silver Bullet`, `Lo esencial y lo accidental en la Ingeniería del Software` (Código Línea), `Guia_ISW_1er_Parcial.docx`.

Tesis Central de Frederick P. Brooks Jr.: *"No existe ningún desarrollo individual, ni en tecnología ni en técnicas de gestión, que por sí solo prometa siquiera una mejora de un orden de magnitud (10 veces) en el plazo de diez años en productividad, confiabilidad o simplicidad"*. Para fundamentarlo, Brooks divide las dificultades del desarrollo de software en dos categorías (siguiendo la tradición filosófica de Aristóteles):

Categoría (Brooks)	Definición Conceptual	Propiedades / Avances Asociados
1. Dificultades Esenciales (Inherentes)	Son las dificultades intrínsecas a la **naturaleza conceptual misma del software**: diseñar, concebir, especificar y articular estructuras conceptuales complejas. *Lo más difícil del software es decidir exactamente qué construir, no codificarlo.*	**Las 4 Propiedades de la Esencia:** 1. **Complejidad:** No hay dos partes iguales; los estados crecen de forma no lineal con el tamaño. 2. **Conformidad:** El software debe adaptarse a leyes, instituciones y sistemas humanos arbitrarios. 3. **Variabilidad (Mutabilidad):** Sufre presiones continuas de modificación porque el entorno cambia y el software se percibe como fácil de alterar. 4. **Invisibilidad:** Es una entidad abstracta e intangible; no tiene representación geométrica espacial única.
2. Dificultades Accidentales (Secundarias)	Son las dificultades asociadas a la **representación física y las herramientas** con las que traducimos esas estructuras conceptuales a código ejecutable por una máquina en un momento histórico dado.	**3 Grandes Avances Pasados que ya eliminaron gran parte del accidente:** 1. **Lenguajes de Alto Nivel** (abstracción sobre ensamblador). 2. **Tiempo Compartido (Time-sharing)** (inmediatez frente al procesamiento por lotes). 3. **Entornos de Programación Unificados** (Unix, IDEs, librerías compartidas).

¿Por qué las "supuestas balas de plata" tecnológicas no logran un salto de 10x? Si las tareas accidentales hoy representan menos de 1/10 del esfuerzo total del proyecto, incluso si una herramienta mágica eliminara el 100% del trabajo accidental (codificación y sintaxis), no reduciría el esfuerzo total en un orden de magnitud porque el 90% restante reside en la **esencia** (comprender el dominio, especificar

requerimientos y diseñar la solución). Por ello, avances como Ada, la Programación Orientada a Objetos, la IA, la programación automática o gráfica aportan mejoras incrementales valiosas pero no son una "bala de plata".

Los 4 Ataques Prometedores contra la Esencia propuestos por Brooks: (1) **Comprar frente a Construir (*Buy vs. Build*):** El modo más radical de no enfrentar la esencia de construir software es adquirir paquetes comerciales o reutilizar componentes ya probados; (2) **Refinamiento de Requisitos y Prototipado Rápido:** Dado que el cliente no sabe con precisión qué quiere hasta que interactúa con algo, iterar con prototipos tempranos ataca la dificultad de definir la estructura conceptual; (3) **Desarrollo Incremental (Hacer crecer el software orgánicamente, no construirlo de golpe):** Inspirado en Harlan Mills, construir primero un esqueleto funcional mínimo que corra de punta a punta e ir agregando funciones progresivamente; y (4) **Grandes Diseñadores (*Great Designers*):** Identificar, formar, mentorizar y retener a los arquitectos y diseñadores con talento excepcional, ya que entre un diseñador mediocre y uno brillante sí existe una diferencia de un orden de magnitud.

1.4 Marco Conceptual Cynefin (Dave Snowden) y Selección del Enfoque

Fuentes de cátedra (Notebook ISW): `Cynefin Mini book.pdf`, `01 Introducción a la Ingenieria de Software.pdf`, `Guia_ISW_1er_Parcial.docx`.

El marco **Cynefin** clasifica los problemas y contextos operativos según la relación entre **causa y efecto** y el nivel de incertidumbre en los requerimientos y la tecnología (Matriz de Stacey), dividiendo el espacio en dominios **Ordenados** (Simple y Complicado), **No Ordenados** (Complejo y Caótico) y un quinto dominio central de **Desorden**:

Dominio Cynefin	Relación Causa-Efecto	Secuencia de Acción	Tipo de Práctica y Proceso de Software
1. Simple / Obvio / Claro (*Clear*)	Conocida, evidente para todos, repetible y predecible (*"Known Knowns"*).	**Percibir – Categorizar – Responder** (*Sense – Categorize – Respond*).	**Mejores Prácticas (*Best Practices*) · Proceso Definido** (procedimientos estándar, recetas fijas).
2. Complicado (*Complicated*)	Existe relación causa-efecto, pero no es evidente a simple vista; requiere análisis de expertos (*"Known Unknowns"*).	**Percibir – Analizar – Responder** (*Sense – Analyze – Respond*).	**Buenas Prácticas (*Good Practices*) · Proceso Definido** con planificación liderada por expertos (ej. Cascada, Iterativo-Incremental tradicional, RUP).
3. Complejo (*Complex*)	Causa y efecto solo se comprenden en retrospectiva; el entorno cambia y los resultados son emergentes (*"Unknown Unknowns"*).	**Sondear (Experimentar) – Percibir – Responder** (*Probe – Sense – Respond*).	**Prácticas Emergentes (*Emergent Practices*) · Proceso Empírico (Ágil: Scrum, Lean, Kanban)** basado en experimentos seguros para fallar (MVP, Sprints).
4. Caótico (*Chaotic*)	No hay relación causa-efecto perceptible; crisis urgente o caída crítica en curso (*"Unknowables"*).	**Actuar – Percibir – Responder** (*Act – Sense – Respond*).	**Prácticas Novedosas (*Novel Practices*)** · Acción inmediata para estabilizar la emergencia y llevar el problema al dominio Complejo.

Dominio Cynefin	Relación Causa-Efecto	Secuencia de Acción	Tipo de Práctica y Proceso de Software
5. Desorden / Confusión (*Disorder*)	Se desconoce en cuál de los otros cuatro dominios se encuentra el problema.	Dividir el problema en partes y asignar cada parte a su dominio real.	Riesgo: cada líder intenta aplicar el método con el que se siente más cómodo si no diagnostica el contexto.

2. Unidad 01 (Continuación): Procesos, Ciclos de Vida y Componentes del Proyecto

Fuentes de cátedra (Notebook ISW): `06 Componentes de Proyecto SW.pdf`, `Diferencias entre Enfoques Tradicionales y Ágiles`, `Scrum y PUD: Ciclos de Vida`, `Guia_ISW_1er_Parcial.docx`.

2.1 Procesos Definidos vs. Procesos Empíricos y Relación con el Ciclo de Vida

- **¿Qué es un Proceso de Desarrollo de Software?** Es el conjunto estructurado de actividades, acciones, tareas, roles, artefactos (entradas/salidas) y restricciones necesarios para transformar los requerimientos del usuario en un producto de software. Sus tres propiedades de evaluación son: **Visibilidad** (determinar cuánto se hizo y cuánto falta mediante resultados tangibles), **Confiabilidad** (medida en que se puede depender del proceso para producir resultados consistentes) y **Modificabilidad** (esfuerzo requerido para adaptar el proceso ante cambios).
- **¿Qué es el Ciclo de Vida del Software (Modelo de Proceso)? Verdad clave de cátedra: *"El ciclo de vida es una representación simplificada de un proceso de desarrollo"*** que establece las fases o etapas y el orden temporal en que se ejecutan desde la concepción hasta la entrega. Mientras que el proceso define el *qué*, *quién* (roles), *cómo* y *con qué artefactos*, el ciclo de vida organiza la secuencia estructural que condiciona directamente cómo se administra y planifica el proyecto.

Dimensión	Proceso Definido (Predictivo / Tradicional)	Proceso Empírico (Adaptativo / Ágil)
Premisa Teórica	Dado un problema bien definido, es posible especificar de antemano todas las actividades paso a paso para obtener salidas **predecibles y repetibles**.	El desarrollo de software tiene alta incertidumbre ("ruido"); el control se logra mediante ciclos cortos de **Transparencia, Inspección y Adaptación**.
Gestión del Cambio	Los requerimientos se relevan al inicio en una ERS detallada y se establecen en una Línea Base; **sí pueden cambiar**, pero a través de un procedimiento formal de Control de Cambios (CCB).	El cambio es bienvenido como ventaja competitiva; el Product Backlog emerge y se reprioriza continuamente al final de cada iteración según el feedback.
Triple Restricción	**Alcance fijo** (determinado por la ERS inicial); el **Tiempo y el Costo son variables** (se estiman en función del alcance).	**Tiempo (Sprints/Fecha) y Costo (Equipo) fijos**; el **Alcance es variable y negociable** (se entrega primero lo de mayor valor).
Cuándo Elegirlo	Requerimientos estables y conocidos desde el inicio, tecnología probada, contratos de alcance cerrado o sistemas regulados críticos.	Requerimientos volátiles o emergentes, productos innovadores de mercado incierto, urgencia por validar valor temprano (time-to-market).

2.2 Comparativa de Modelos de Ciclo de Vida (Secuenciales, Iterativos e Incrementales)

Modelo de Ciclo de Vida	Funcionamiento y Estructura	Ventajas Principales	Desventajas yCuándo Evitarlo
Cascada Pura (Secuencial Lineal)	Ejecución secuencial estricta: Requerimientos → Diseño → Codificación → Pruebas → Implementación. Cada fase requiere cerrarse y aprobarse antes de iniciar la siguiente.	Simple de entender y gestionar; hitos y entregables documentales muy claros en cada fase.	**Baja visibilidad de software funcionando hasta el final**; rigidez ante cambios; alto riesgo de descubrir tarde que el producto no era lo que el usuario quería.
Prototipado Evolutivo / Desechable	Construcción rápida de un modelo funcional para que el usuario experimente con la interfaz y clarifique requerimientos ambiguos.	Excelente para reducir incertidumbre de requisitos y validar usabilidad temprano con el cliente.	El cliente puede confundir el prototipo con el sistema terminado o presionar para poner en producción código rápido sin calidad arquitectónica.
Entrega por Etapas (Incremental Planificado)	Se especifican los requerimientos y el diseño arquitectónico global al inicio, pero la construcción y entrega se dividen en etapas sucesivas planificadas.	Permite entregar funcionalidad útil en plazos más cortos manteniendo la previsibilidad de un proceso definido.	Requiere conocer bien los requerimientos globales y la arquitectura base al principio del proyecto.
Iterativo e Incremental (RUP / PUD / Ágil)	El sistema crece por **iteraciones** (refinamiento progresivo de la calidad/comprensión) e **incrementos** (agregado de nuevas porciones funcionales completas).	Feedback continuo del usuario, mitigación temprana de riesgos técnicos, entregas frecuentes de valor y adaptación al cambio.	Exige alto involucramiento del cliente/PO, disciplina técnica (SCM, pruebas de regresión) y gestión activa del alcance.
Espiral (Barry Boehm)	Modelo evolutivo impulsado por el riesgo; cada vuelta de la espiral atraviesa 4 cuadrantes: (1) Determinar objetivos, (2) **Identificar y resolver riesgos** (prototipos), (3) Desarrollar y probar, (4) Planificar la siguiente iteración.	Es el único modelo que incorpora explícitamente el **análisis de riesgos en cada iteración**; ideal para sistemas grandes, complejos y de alto riesgo.	Costoso de administrar; requiere expertos en evaluación de riesgos y es difícil de aplicar en contratos a precio fijo.

Nota sobre el Proceso Unificado de Desarrollo (PUD / RUP): Es un proceso iterativo e incremental, dirigido por casos de uso, centrado en la arquitectura y enfocado en los riesgos. Se estructura bidimensionalmente en **4 Fases dinámicas** a lo largo del tiempo (*Inicio / Inception*, *Elaboración*, *Construcción* y *Transición*) y **Disciplinas estáticas** (Modelado de Negocio, Requerimientos, Análisis y Diseño, Implementación, Pruebas, Despliegue, más soporte de SCM, Gestión de Proyectos y Entorno) que se ejecutan con distinta intensidad en cada iteración.

2.3 Componentes de un Proyecto de Software (Las 4 P) y Vínculo Proceso-Proyecto-Producto

- **¿Qué es un Proyecto?** Es un esfuerzo **temporal** (tiene inicio y fin definidos) que se lleva a cabo para crear un producto, servicio o resultado **único**, orientado a objetivos y desarrollado mediante

elaboración gradual.

- **Vínculo entre Proceso, Proyecto, Producto, Personas y Herramientas (Pregunta Clásica de Parcial):**
  - El **Proceso** define en forma teórica el marco de actividades, roles y artefactos.
  - Ese proceso se instancia y adapta en un **Proyecto** (unidad organizativa temporal con recursos, plazos y un ciclo de vida elegido).
  - Las **Personas** (el factor más crítico) ejecutan las actividades del proceso dentro del proyecto apoyándose en **Herramientas**.
  - El resultado generado por el proyecto es una versión del **Producto** (software, código, datos y documentación).
- **Diferencia entre Ciclo de Vida del Producto vs. Ciclo de Vida del Proyecto (Regla de Oro de Cátedra):** El **Ciclo de Vida del Producto** es mucho más amplio: **comienza con la idea o concepción inicial de negocio y termina recién con el retiro o discontinuación definitiva del software en el mercado** (abarca introducción, crecimiento, madurez y declive). En cambio, el **Ciclo de Vida del Proyecto** es temporal (inicio, planificación, ejecución, control y cierre) y su objetivo es construir o evolucionar **una versión o release específico** de ese producto. Un mismo producto de software suele atravesar múltiples proyectos a lo largo de su vida útil.
- **La Triple Restricción (Triángulo de Hierro):** Relaciona **Alcance (Funcionalidad)**, **Tiempo (Cronograma)** y **Costo (Presupuesto/Recursos)** con la **Calidad** en el centro. No es posible fijar arbitrariamente las tres variables a la vez sin degradar la calidad: si el cliente fija Fecha y Presupuesto, el Alcance debe ser variable y priorizado; la calidad nunca es negociable.

3. Unidad 02 (Parte I): Gestión de Producto, Requerimientos Ágiles, Estimaciones y Scrum

Fuentes de cátedra (Notebook ISW): `05 Gestión de Productos.pdf`, `03 Rqs Agiles USER STORIES.pdf`, `04 Estimaciones Ágiles.pdf`, `07 Estimaciones de Sw.pdf`, `08 SCRUM 2026, Planificación de release y sprint, Métricas Scrum.pdf`, `Guia_ISW_1er_Parcial.docx`.

3.1 Gestión de Productos de Software: Discovery vs. Delivery, MVP, MMP y MMR

Según el Standish Group, apenas el **7% de las funcionalidades** construidas en los sistemas tradicionales se usan "siempre" y el **45% no se usa nunca**. Por ello, la gestión moderna de productos se enfoca en maximizar el **Outcome (valor e impacto real logrado en el usuario/negocio)** por encima del simple **Output (cantidad de código o funciones construidas)**, articulando dos flujos continuos: **Product Discovery** (descubrir *qué* construir validando si es valioso, usable y factible) y **Product Delivery** (construir y entregar el software con calidad de producción).

Concepto	Significado Completo	Objetivo Estratégico y Diferencia Clave en Parcial
UVP	*Unique Value Proposition* (Propuesta de Valor Única)	Declaración e hipótesis inicial que explica qué problema resuelve el producto, para qué perfil de usuario y por qué es distinto o superior a las alternativas.

Concepto	Significado Completo	Objetivo Estratégico y Diferencia Clave en Parcial
MVP	*Minimum Viable Product* (Producto Mínimo Viable)	Es la versión o **experimento más pequeño** que permite recorrer el ciclo **Construir – Medir – Aprender** (Lean Startup) con el menor esfuerzo para **validar o refutar una hipótesis de negocio** en usuarios reales y decidir si *perseverar* o *pivotar*. Incluye solo la funcionalidad núcleo indispensable de punta a punta (metáfora de Kniberg: patineta funcional, no una rueda suelta).
MMF / MVF	*Minimum Marketable / Viable Feature*	La funcionalidad individual más pequeña y autocontenida capaz de generar valor concreto y percibido por sí misma para el cliente.
MMP (= MMR 1)	*Minimum Marketable Product* / Primer *Minimum Marketable Release*	Es el **primer lanzamiento comercializable** listo para salir al mercado general o *early adopters* con calidad de producción y las MMFs necesarias para cobrar o operar formalmente. **Regla de oro de cátedra: MVP ≠ MMP.** El objetivo del MVP es **aprender** (validar la hipótesis); el objetivo del MMP es **comercializar/vender**. Lo aprendido con el MVP informa qué debe incluir el MMP.

3.2 Requerimientos Ágiles y User Stories (Historias de Usuario)

- **Niveles de Granularidad en el Product Backlog:**
  1. **Tema (*Theme*):** Agrupación estratégica de alto nivel que reúne varias épicas o historias vinculadas a un mismo objetivo o área funcional del producto (ej. *"Gestión de Cobros y Facturación"* o *"Seguimiento en vivo de la carrera"*). **¡Alerta de parcial!:** Un filtro o un botón nunca es un Tema.
  2. **Épica (*Epic*):** Historia de usuario demasiado grande, compleja o con demasiada incertidumbre como para ser completada en un único Sprint; debe dividirse (*slicing*) en historias más pequeñas antes de entrar a un Sprint.
  3. **User Story (Historia de Usuario):** Descripción breve y simple de una funcionalidad contada desde la perspectiva del usuario o cliente que desea esa nueva capacidad. Representa una **porción vertical (*vertical slice*)** que atraviesa todas las capas arquitectónicas (interfaz gráfica, lógica de negocio y persistencia de datos) entregando valor funcional verificable de principio a fin.
  4. **Tarea Técnica (*Task*):** Unidad de trabajo técnico en que los Developers descomponen una User Story durante el Sprint Planning (ej. *"Crear tabla SQL"*, *"Codificar endpoint REST"*); se estima en horas y no entrega valor de negocio por sí sola.

- **Las 3 C de una User Story (Ron Jeffries) y Formato de la Tarjeta:**
  - **1. Tarjeta (*Card*):** Recordatorio físico o digital escrito que contiene la **Frase Verbal** con la plantilla de Mike Cohn: **`COMO [rol de usuario concreto] QUIERO [actividad / funcionalidad que hará el sistema] PARA [valor de negocio / beneficio que obtiene]`**, acompañada de una breve descripción o reglas de negocio. *Las User Stories NO son especificaciones exhaustivas de requerimientos, sino un recordatorio (token) para conversar.*
  - **2. Conversación (*Conversation*):** El diálogo continuo cara a cara entre el Product Owner y los Developers durante el refinamiento y el Sprint para aclarar detalles; es el componente más importante.
  - **3. Confirmación (*Confirmation*):** Los **Criterios de Aceptación** y las **Pruebas de Aceptación / Usuario** que definen los límites de la historia y permiten al Product Owner verificar objetivamente si

la historia está completa y funciona según lo esperado.

- **El Modelo INVEST (Bill Wake) — Calidad de una User Story dentro del *Definition of Ready (DoR)*:**
  - **I — Independent (Independiente):** Debe poder planificarse e implementarse en cualquier orden sin depender fuertemente de otra historia.
  - **N — Negotiable (Negociable):** No es un contrato cerrado; los detalles se negocian en la conversación entre el PO y los Developers.
  - **V — Valuable (Valiosa):** Debe aportar valor explícito y comprensible a un usuario o al cliente (por eso no existen "User Stories técnicas" sin valor para el usuario).
  - **E — Estimable (Estimable):** El equipo comprende lo suficiente sobre el dominio y la técnica como para poder estimar su tamaño en Story Points (si no es estimable por alta incertidumbre, se realiza antes un **Spike**).
  - **S — Small (Pequeña / *Sized Appropriately*):** Su tamaño debe permitir completarla íntegramente (cumpliendo la DoD) dentro de un único Sprint.
  - **T — Testable (Testeable / Comprobable):** Debe contar con criterios de aceptación claros y medibles (evitando adverbios vagos como *"rápidamente"* o *"fácilmente"* sin métrica).
- **¿Qué es un Spike (Historia de Investigación / Exploración)?** Es una actividad con un **límite de tiempo fijo (*timebox*)** ejecutada en una **iteración previa** cuyo único propósito es investigar y **reducir la incertidumbre técnica** (*Spike Técnico*, ej. evaluar si una librería de streaming soporta baja latencia) o **funcional** (*Spike Funcional*, ej. validar con un prototipo cómo interactúa el usuario). Su entregable es conocimiento o una prueba de concepto que permite que la User Story original vuelva a ser **Estimable (INVEST)** en el Sprint siguiente.

3.3 Estimaciones de Software: Enfoque Tradicional vs. Enfoque Ágil

El **Cono de Incertidumbre (Barry Boehm)** demuestra que al inicio del proyecto las estimaciones pueden variar hasta en un factor de ±4× (400%) y que la incertidumbre solo disminuye a medida que se toman decisiones y avanza el proyecto. Asimismo, la **Ley de Brooks** establece que *"añadir más personas a un proyecto de software atrasado solo lo atrasará más"* debido al crecimiento combinatorio de los canales de comunicación (n(n−1)/2) y el tiempo de capacitación.

Criterio (No mezclar en el parcial)	Estimación Tradicional (Proceso Definido)	Estimación Ágil (Proceso Empírico / Scrum)
Secuencia de lo que se estima	**1º Tamaño** del producto → **2º Esfuerzo** → **3º Calendario (Duración)** → **4º Costo y Recursos críticos**. *(La estimación de esfuerzo se deriva de la de tamaño, nunca al revés).*	**Del Producto:** Tamaño relativo de cada User Story del Product Backlog. **Del Proyecto (Release):** Cantidad de Sprints / Fecha y Costo se **derivan** dividiendo los Story Points totales por la **Velocidad** del equipo.
Unidades de Medida	**Tamaño:** Puntos de Función (FP) o Líneas de Código (LOC). **Esfuerzo:** Horas-persona, Días-hombre o Meses-persona.	**User Stories: Puntos de Historia (*Story Points - SP*)** con serie **Fibonacci** (1, 2, 3, 5, 8, 13, 21…). **Tareas del Sprint Backlog:** Horas ideales/reales.

Criterio (No mezclar en el parcial)	Estimación Tradicional (Proceso Definido)	Estimación Ágil (Proceso Empírico / Scrum)
Qué contempla la unidad	Esfuerzo técnico estimado a partir de datos históricos, descomposición WBS o modelos algorítmicos (COCOMO), más margen de contingencia por riesgos.	Un **Story Point** es una medida relativa y propia de cada equipo que combina **3 componentes obligatorios: (1) Complejidad, (2) Esfuerzo / Volumen de trabajo y (3) Incertidumbre / Riesgo**.
Quién estima y con qué técnica	El **Líder de Proyecto** junto a expertos técnicos mediante juicio de expertos, analogía histórica o **Wideband Delphi**.	**Todo el equipo de Developers** que construirá el producto mediante **Planning Poker** y comparación por triangulación contra una **User Story Canónica (1 SP o 2 SP)**.
Cuándo se estima	Al inicio del proyecto tras especificar la ERS (para armar el Plan de Proyecto) y se re-estima en los hitos de control de fase.	Al inicio para el Release Plan y de forma continua en el **Refinamiento del Product Backlog** y en el **Sprint Planning**.

Pasos de la Técnica *Planning Poker* (Orden exacto evaluado): (1) El Product Owner lee la User Story y explica su valor y criterios de aceptación; (2) El equipo conversa con el PO y hace preguntas para aclarar dudas técnicas y funcionales; (3) Cada Developer elige en secreto su carta con el valor Fibonacci que representa su estimación en Story Points; (4) Todos muestran sus cartas simultáneamente (para evitar el sesgo de anclaje); (5) Si hay divergencia, los estimadores con el valor más alto y el más bajo exponen sus argumentos; (6) Se vuelve a votar hasta alcanzar consenso.

3.4 Framework Scrum (Guía Oficial Scrum 2020 y Enfoque de Cátedra 2026)

Definición Oficial: Scrum es un marco de trabajo liviano (*framework*) que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos. Se basa en el **Empirismo** (el conocimiento proviene de la experiencia y de tomar decisiones basadas en lo que se observa) y el **Pensamiento Lean** (reducir el desperdicio y enfocarse en lo esencial).

- **Los 3 Pilares del Control de Procesos Empíricos en Scrum:**
  1. **Transparencia:** El proceso y el trabajo emergente deben ser visibles tanto para quienes realizan el trabajo como para quienes lo reciben; los artefactos tienen un estándar compartido (como la *Definition of Done*).
  2. **Inspección:** Los artefactos de Scrum y el progreso hacia los objetivos acordados deben inspeccionarse con frecuencia y diligencia en los 5 eventos para detectar variaciones o problemas indeseados.
  3. **Adaptación:** Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso o los materiales deben ajustarse lo antes posible.
- **Los 5 Valores de Scrum: Compromiso** (*Commitment*), **Foco / Enfoque** (*Focus*), **Franqueza / Apertura** (*Openness*), **Respeto** (*Respect*) y **Coraje / Valor** (*Courage*).

Responsabilidades (Roles) del Scrum Team (10 o menos personas, multifuncional y autogestionado)

Responsabilidad (Rol)	Funciones Clave y Qué NO debe hacer (según Cátedra ISW)
Product Owner *(1 sola persona humana, no un comité)*	• Es el único responsable de **maximizar el valor del producto** resultante del trabajo del Scrum Team. • Gestiona de forma efectiva el **Product Backlog**: desarrolla y comunica explícitamente el *Product Goal*, crea y comunica los ítems, los **ordena/prioriza por valor** (tiene la última palabra sobre las prioridades) y asegura que sea transparente y visible. • Define el *"qué"* y el *"por qué"*. **NO asigna tareas a los Developers ni decide cómo se construye técnicamente ni delega el Product Backlog en el Scrum Master.**
Scrum Master *(Líder servicial que gestiona el proceso)*	• Es responsable de establecer Scrum tal como se define en la Guía de Scrum y de la **efectividad del Scrum Team**. • Sirve al Equipo Scrum (guiando en autogestión y multifuncionalidad, ayudando a enfocarse en crear incrementos de alto valor que cumplan la DoD, causando la **eliminación de impedimentos** y asegurando que todos los eventos sean positivos, productivos y respeten su *timebox*), sirve al Product Owner y sirve a la Organización. • **NO es un "Líder de Proyecto" tradicional** (ese rol no existe en Scrum) ni rechaza historias en la Review.
Developers *(Profesionales que crean el incremento)*	• Son las personas del Scrum Team comprometidas a crear cualquier aspecto de un Incremento utilizable en cada Sprint (programadores, diseñadores, testers, analistas, arquitectos, sin sub-equipos ni jerarquías). • Crean el plan para el Sprint (**Sprint Backlog**), inculcan calidad adhiriéndose a la **Definition of Done**, adaptan su plan cada día hacia el Sprint Goal y se responsabilizan mutuamente como profesionales.

Los 5 Eventos de Scrum y sus Timeboxes (Para un Sprint de 4 semanas / 1 mes)

Evento	Timebox Máximo	Quiénes Participan	Propósito de Inspección y Adaptación
1. El Sprint	**1 mes o menos** (fijo y constante).	Todo el Scrum Team.	Evento contenedor de todos los demás eventos donde las ideas se convierten en valor. Durante el Sprint: no se hacen cambios que pongan en peligro el *Sprint Goal*, la calidad no disminuye y el alcance puede renegociarse con el PO a medida que se aprende. **Solo el Product Owner puede cancelar un Sprint** si el Sprint Goal queda obsoleto.
2. Sprint Planning	**Máximo 8 horas** (para Sprint de 1 mes).	Scrum Team completo (pueden invitar a expertos).	Inicia el Sprint respondiendo **3 temas**: (1) *¿Por qué es valioso este Sprint?* (se define el **Sprint Goal**); (2) *¿Qué se puede hacer en este Sprint?* (Developers seleccionan ítems del Product Backlog según su velocidad y capacidad); (3) *¿Cómo se realizará el trabajo elegido?* (Developers descomponen las US en tareas técnicas de ≤ 1 día). Resultado: **Sprint Backlog**.

Evento	Timebox Máximo	Quiénes Participan	Propósito de Inspección y Adaptación
3. Daily Scrum	**Exactamente 15 minutos** diarios.	**Developers** (si PO o SM trabajan en ítems del Sprint, participan como Developers).	Inspeccionar el progreso hacia el **Sprint Goal** y adaptar el Sprint Backlog para las próximas 24 horas. Reduce la complejidad y evita otras reuniones; los impedimentos profundos se tratan fuera de los 15 minutos.
4. Sprint Review	**Máximo 4 horas** (para Sprint de 1 mes).	Scrum Team + **Stakeholders clave** invitados por el PO.	Inspeccionar el resultado del Sprint (**Incremento funcionando** que cumple la DoD, nunca diapositivas ni historias incompletas) y determinar futuras adaptaciones en el **Product Backlog** colaborando con los interesados sobre qué hacer a continuación.
5. Sprint Retrospective	**Máximo 3 horas** (para Sprint de 1 mes).	Scrum Team completo.	Planificar formas de aumentar la calidad y la efectividad inspeccionando cómo fue el último Sprint respecto a las **personas, interacciones, procesos, herramientas y la Definition of Done**. Cierra el Sprint.

Nota sobre el Refinamiento del Product Backlog (Backlog Grooming): No es un evento oficial con timebox único, sino una actividad continua en la que el PO y los Developers detallan, dividen y estiman ítems futuros (suele insumir hasta el 10% de la capacidad del Sprint) para que cumplan el criterio de estar listos (**Definition of Ready - DoR**).

Los 3 Artefactos de Scrum, sus Compromisos y la Diferencia DoR vs. DoD

- **1. Product Backlog (Lista de Producto) → Compromiso: *Product Goal (Objetivo del Producto)*:** Lista emergente y ordenada de lo que se necesita para mejorar el producto; es la única fuente de trabajo del Scrum Team. El *Product Goal* describe un estado futuro del producto que sirve como objetivo a largo plazo para planificar.
- **2. Sprint Backlog (Lista de Pendientes del Sprint) → Compromiso: *Sprint Goal (Objetivo del Sprint)*:** Compuesto por el *Sprint Goal* (el "porqué"), el conjunto de ítems del Product Backlog seleccionados para el Sprint (el "qué") y un plan de acción para entregar el Incremento (el "cómo"). Pertenece exclusivamente a los Developers.
- **3. Incremento (*Increment*) → Compromiso: *Definition of Done (Definición de Terminado / Hecho)*:** Peldaño concreto de valor utilizable y aditivo hacia el Product Goal. En el momento en que un ítem del Product Backlog cumple con la **Definition of Done (DoD)**, nace un Incremento. Si un ítem no cumple la DoD al final del Sprint, **no se presenta en la Sprint Review ni suma Story Points a la velocidad**, sino que vuelve al Product Backlog.
- **Distinción Crítica de Parcial: *Definition of Ready (DoR)* vs. *Definition of Done (DoD)* vs. *Criterios de Aceptación*:**
  - **Definition of Ready (DoR):** Acuerdo del equipo sobre las condiciones que debe cumplir una User Story en el Product Backlog **ANTES de ingresar al Sprint Planning** (aquí entra **INVEST**: que sea independiente, negociable, valiosa, estimada en SP, pequeña y con criterios de aceptación claros). **¡Regla de cátedra: INVEST ∈ DoR, INVEST ∉ DoD!**

- **Definition of Done (DoD):** Estándar formal y transversal de calidad técnica y funcional aplicable a **TODOS los ítems del Incremento DESPUÉS de construirlos** (ej. código revisado por pares, cobertura de pruebas unitarias ≥ 80%, pruebas de aceptación aprobadas, sin bugs críticos, integrado en rama principal con CI en verde y desplegado en staging).
- **Criterios de Aceptación:** Condiciones funcionales y no funcionales **específicas de una User Story particular** (ej. validar tramos de comisión o formatos de un campo puntual).

4. Unidad 03 (Parte I): Administración de Configuración de Software (SCM)

Fuentes de cátedra (Notebook ISW): `02 SCM.pdf`, `Agile SCM.pdf`, `Guia_ISW_1er_Parcial.docx`, `ISW_Enunciados TP evaluables 2026_2do Cuatrimestre.docx.pdf`.

4.1 Conceptos Fundamentales de SCM: IC, Versión, Variante, Configuración y Línea Base

La **Administración de Configuración de Software (SCM / ACS)** es una disciplina de soporte y aseguramiento de calidad (SQA) que aplica dirección técnica y administrativa para: (1) identificar y documentar las características funcionales y físicas de los ítems de configuración, (2) controlar los cambios a esas características a lo largo de todo el ciclo de vida, (3) registrar e informar el estado del procesamiento de los cambios, y (4) verificar el cumplimiento de los requisitos especificados. Su propósito es **mantener la integridad y trazabilidad del producto** evitando los 4 problemas clásicos: simultaneidad (pisar cambios de otro), código compartido sin control, versiones múltiples perdidas y falta de comunicación.

- **Ítem de Configuración de Software (IC / SCI):** Cualquier artefacto o unidad de información creada durante el proceso de software que cambia a lo largo del tiempo, es compartida por más de una persona y cuyo estado y evolución necesitamos controlar. Según la cátedra se clasifican en:
  - **ICs de Producto:** Especificación de Requerimientos (ERS) o Product Backlog, Documento de Arquitectura y Diseño, Código Fuente, Casos de Prueba, Manual de Usuario. *(Regla de nombrado de cátedra: los ICs de producto NO llevan fecha ni versión en el nombre del archivo porque el versionado lo gestiona la herramienta SCM).*
  - **ICs de Proyecto:** Plan de Proyecto, Plan de SCM (que es en sí mismo un IC), Cronograma, Plan de Riesgos, Minutas de Reunión (estas sí llevan fecha `ddmm`).
  - **ICs de Iteración:** Plan de Iteración / Sprint Backlog, Notas de Release, Reportes de Defectos de la iteración (llevan el número de iteración).
- **Versión vs. Variante vs. Configuración vs. Repositorio:**
  - **Versión (Revisión):** Instancia o estado de un Ítem de Configuración en un momento determinado de su evolución histórica lineal (la versión v_2 reemplaza o corrige a la v_1).
  - **Variante:** Instancia de un Ítem de Configuración que coexiste en paralelo con otras para adaptarse a diferentes plataformas, idiomas o segmentos de clientes sin reemplazar a la original (ej. variante Android vs. variante iOS; se gestiona con ramas / *branches*).
  - **Configuración (Versión del Producto / Release):** Conjunto consistente y completo de versiones específicas de todos los Ítems de Configuración que conforman el producto o sistema en un instante dado.
  - **Repositorio:** Base de datos o almacén centralizado/distribuido gestionado por una herramienta SCM (como Git) que resguarda los ICs, su historial de versiones y sus metadatos.

- **Línea Base (*Baseline* — Definición textual de cátedra):** *"Es una especificación o producto (conjunto de Ítems de Configuración) que ha sido **revisado formalmente y sobre el cual se ha llegado a un acuerdo**, que de ahí en adelante sirve como base para un desarrollo posterior y que **puede cambiarse solamente a través de procedimientos formales de control de cambios**"*. Representa un hito técnico consolidado (en Git se materializa con un **Tag anotado** como `lb-parcial-01` sin alterar el nombre de los archivos).
  - **Criterio de Línea Base en Proceso Definido:** Al finalizar y aprobar cada fase del ciclo de vida (Línea Base de Requerimientos tras aprobar la ERS, Línea Base de Diseño, Línea Base de Producto al superar pruebas de sistema).
  - **Criterio de Línea Base en Proceso Empírico (Scrum):** Al finalizar cada Sprint (sobre el Incremento que cumple la DoD y es aceptado en la Sprint Review) y en cada Release puesto en producción.

4.2 Las 4 Actividades Principales de la Gestión de Configuración

Actividad de SCM	Descripción Técnica y Procedimiento
1. Identificación de la Configuración	Seleccionar qué artefactos serán ICs, clasificarlos (producto, proyecto, iteración), definir la estructura de carpetas del repositorio (en `snake_case`), establecer las **reglas de nombrado** unívocas (en `kebab-case`) y fijar los criterios para establecer las Líneas Base.
2. Control de Versiones y Control de Cambios	Administrar la evolución de los ICs y regular las modificaciones sobre las Líneas Base para evitar el caos: **Flujo Formal de Control de Cambios:** (1) Un interesado envía una **Solicitud de Cambio (RFC)**; (2) Se realiza la evaluación técnica y el **Análisis de Impacto** (costo, plazo, módulos afectados, riesgo); (3) El **Comité de Control de Cambios (CCB — *Change Control Board*)**, integrado por representantes con distintas visiones (negocio/cliente, proyecto, arquitectura, calidad/SCM), evalúa el informe y **aprueba, rechaza o posterga** el cambio; (4) Si se aprueba, se genera una Orden de Cambio (ECO), se hace *check-out* de los ICs, se implementa y prueba la modificación, se audita y se hace *check-in* creando una nueva versión y actualizando la Línea Base.
3. Auditoría de la Configuración	Examen independiente que garantiza que se mantuvo la integridad de la Línea Base: • **Auditoría Funcional (FCA):** Verifica mediante resultados de pruebas y matrices de trazabilidad que el software construido cumple todas las funciones y rendimientos especificados en la ERS. • **Auditoría Física (PCA):** Verifica que todos los ICs físicos (código, binarios, manuales, scripts, documentos) estén presentes en el repositorio en su versión exacta, completos y coherentes antes de la liberación.
4. Generación de Informes de Estado (Contabilidad)	Registrar y reportar qué cambios se solicitaron, quién los aprobó, quién los implementó, cuándo se realizaron y qué versiones componen cada Línea Base, asegurando que la información llegue a todos los involucrados.

SCM en Entornos Ágiles y Operaciones con Git: En metodologías ágiles **sí existe SCM y es aún más crítica** porque los cambios e integraciones ocurren todos los días. En lugar de un CCB burocrático lento, el control de cambios se agiliza mediante la priorización del Product Backlog por el Product Owner, la **Integración Continua (CI)**, pruebas automatizadas, revisiones por pares (*Pull Requests*) y herramientas distribuidas como **Git** (operaciones clave: `clone`, `add`, `commit`, `push`, `pull`, creación de ramas con `branch`, fusión con `merge` o `rebase` y etiquetado de líneas base con `git tag -a lb-parcial-01`).

5. Guía Práctica de Resolución del Primer Parcial y las 12 Claves de Cátedra

Fuentes de cátedra (Notebook ISW): `Guia_ISW_1er_Parcial.docx`, `1er parcial ISW 9-05-25.pdf`, `Primer parcial 4k4 2025 - Nota 7 - MVP perfecto.pdf`.

5.1 Algoritmo Paso a Paso para Resolver el Ejercicio Práctico de Product Owner (50 Puntos)

1. **Parte 1 — Definir el Mínimo Producto Viable (MVP) (18 puntos):**
   - **a) Enunciar el Objetivo del MVP y la Hipótesis de Negocio a Validar:** Redactar explícitamente qué supuesto crítico de negocio queremos comprobar con usuarios reales (ej. *"Validar la hipótesis de que los usuarios están dispuestos a registrarse, publicar artículos y concretar intercambios solidarios a través de la plataforma..."*).
   - **b) Listar las User Stories que integran el MVP (usando SOLO su Frase Verbal):** Seleccionar únicamente las historias esenciales que conforman un flujo funcional completo de punta a punta (por ejemplo: registrarse/iniciar sesión, cargar la entidad principal, buscar/visualizar, ejecutar la transacción núcleo del negocio y consultar el resultado básico).
   - **c) Justificar qué entra en el Alcance del MVP y qué queda FUERA del MVP (¡Obligatorio!):** Explicar el criterio de selección (menor esfuerzo para aprender y validar el corazón del negocio) y **nombrar explícitamente las funcionalidades del enunciado que se postergan para futuros releases (MMP / MMR2)** (como reportes estadísticos históricos, pasarelas de pago secundarias, notificaciones push avanzadas, redes sociales o paneles administrativos complejos), explicando que no son indispensables para validar la hipótesis inicial.

2. **Parte 2 — Redactar la Tarjeta Completa de la User Story Solicitada y su Estimación (32 puntos):**
   - **a) Leer con precisión qué requerimiento pide la consigna:** No elegir cualquier historia del dominio, sino exactamente la que pide el enunciado.
   - **b) Frase Verbal y Descripción:** Redactar `COMO [rol] QUIERO [acción] PARA [valor de negocio]` y debajo un párrafo breve con los datos o contexto de negocio de la historia.
   - **c) Criterios de Aceptación (Completos y Exhaustivos):** Enumerar todas las condiciones que el sistema debe verificar: (i) campos obligatorios y opcionales con sus formatos; (ii) reglas de negocio, cálculos o tramos numéricos exactos del enunciado (con sus bordes <, ≤, >, ≥); (iii) filtros o estados válidos; (iv) mensajes ante casos vacíos o errores de validación; y (v) **Requerimientos No Funcionales (RNF)** vinculados a esa historia (ej. tiempo de respuesta ≤ 2 segundos, precisión de 2 decimales).
   - **d) Pruebas de Usuario / Aceptación:** Escribir escenarios concretos con **valores específicos** (nunca genéricos): incluir al menos **una prueba que "PASA" por cada combinación o tramo válido distinto** y pruebas que "FALLAN / RECHAZAN" para cada regla de validación inválida.
   - **e) Elegir y Justificar la User Story Canónica (1 SP o 2 SP):** Escribir la frase verbal de la historia más simple, conocida y de menor incertidumbre del MVP (ej. un alta o consulta simple) y asignarle 1 SP (o 2 SP) como unidad base de referencia.
   - **f) Estimar la User Story principal en Story Points (escala Fibonacci) justificando sus 3 componentes:** Asignar un valor Fibonacci (ej. 5 SP u 8 SP) y justificar por separado: (1) **Complejidad** (validaciones, reglas, cálculos o integraciones que tiene), (2) **Esfuerzo / Volumen** (cantidad de pantallas, campos y pruebas respecto a la canónica) y (3) **Incertidumbre / Riesgo** (**¡Nunca poner incertidumbre "nula" si hay integraciones externas o reglas nuevas!**), concluyendo por qué equivale a X veces la US Canónica.

5.2 Resumen de las 12 "Verdades" de Corrección que más Puntos Dan o Quitan en el 1er Parcial

#	Error Frecuente Penalizado en Parciales	Criterio Correcto Exigido por la Cátedra ISW
1	Definir ciclo de vida como "los pasos sin decir quién lo hace".	**El ciclo de vida es una representación simplificada del proceso** (fases y su orden) que condiciona la administración del proyecto.
2	Decir que el ciclo de vida del producto "empieza con el MVP".	El **CV del producto empieza con la idea y termina con el retiro**; el CV del proyecto es temporal y produce una versión.
3	Responder que "Sí, el MVP es igual al MMP (o primer release)".	**MVP ≠ MMP.** El MVP es un experimento para **aprender** (validar hipótesis); el MMP es para **comercializar**. El MVP informa al MMP.
4	Decir que el modelo INVEST forma parte de la *Definition of Done (DoD)*.	**INVEST ∈ DoR (Definition of Ready) e INVEST ∉ DoD.** INVEST evalúa si la US está lista *antes* de construir; la DoD evalúa el Incremento *después* de construir.
5	Nombrar un filtro o acción puntual como si fuera un "Tema".	Un **Tema** es un área funcional amplia que agrupa épicas e historias; un filtro suele ser parte de los criterios de aceptación de una US de consulta.
6	Mezclar estimación tradicional y ágil en la misma respuesta.	Mantener coherencia estricta: **Tradicional** = FP/LOC → Horas-persona / Líder + Expertos / tras la ERS. **Ágil** = Story Points (Fibonacci) / Developers / Planning Poker.
7	Definir un Spike de forma vaga sin sus elementos obligatorios.	Todo **Spike** debe indicar: **tipo** (técnico/funcional), **incertidumbre que resuelve**, **timebox fijo**, que se hace en una **iteración previa** y su **resultado** (volver estimable la US).
8	Armar el MVP listando solo lo que entra sin mencionar lo que queda afuera.	Es obligatorio **justificar explícitamente el alcance NO contemplado** (qué funcionalidades del enunciado quedan fuera del MVP y por qué).
9	Escribir Criterios de Aceptación incompletos o genéricos.	Detallar todas las opciones, tramos numéricos, límites exactos, RNF aplicables, mensajes de error y comportamiento ante búsquedas vacías.
10	Escribir Pruebas de Usuario sin datos concretos.	Incluir **valores concretos de ejemplo** y al menos **una prueba que "Pasa" por cada combinación o tramo válido distinto** más las que "Fallan".
11	Declarar "Incertidumbre nula" al justificar los Story Points.	La incertidumbre nunca es nula en historias con reglas múltiples o integraciones; justificar siempre Complejidad + Esfuerzo + Incertidumbre frente a la Canónica.
12	Responder en abstracto sin ejemplificar con el dominio del examen.	Toda pregunta teórica de desarrollo aplicada al caso resta entre 30% y 50% si no se ejemplifica con roles y funciones concretas del enunciado del parcial.